



Bulut Bilişim Ve Uygulamaları

BLM(3522)

**BULUT TABANLI GÜVENLİK KAMERASI İHLAL
TESPİT SİSTEMİ - PROJE GELİŞTİRME
DOKÜMANTASYONU**

Hazırlayan: Berkay Bozlar

Numara: 23290939

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar Mühendisliği

GitHub:(<https://github.com/BerkayBozlar/GuvenlikIhlalSistemi>)

Tanıtım Videosu:

(https://drive.google.com/file/d/1T653voBSZ9hyPYhK3RHjREllxz2HFukE/view?usp=drive_link)

Projenin temel amacı, güvenlik kameralarından elde edilen video akışlarını bulut ortamına aktararak, yapay zeka destekli görüntü işleme servisleri aracılığıyla videodaki nesneleri ve kişileri (insan, araç vb.) anlık olarak analiz eden bir sistem kurmaktır. Bu doğrultuda, Python programlama dili ve Flask web çatısı (framework) kullanılarak AWS Rekognition API'si ile entegre çalışan uçtan uca bir web uygulaması geliştirilmiştir.

BÖLÜM 1: PROJENİN BAŞLATILMASI VE AWS GÜVENLİK ALTYAPISININ KURULMASI

Uygulamanın kodlanmasına geçilmeden önce, projenin üzerinde çalışacağı bulut bilişim altyapısının (Depolama ve Yapay Zeka servisleri) güvenli bir şekilde ayağa kaldırılması sağlanmıştır.

1. Proje Geliştirme Ortamı

- **Kullanılan IDE:** Visual Studio Code (VS Code)
- **Programlama Dili & Framework:** Python 3.x & Flask
- **Kullanılan AWS Servisleri:** Amazon S3 (Depolama), Amazon Rekognition (Video Analizi)
- **Proje Adı:** GuvenlikIhlalSistemi

2. AWS IAM (Kimlik ve Erişim Yönetimi) Yapılandırması Geliştirilecek olan Python uygulamasının AWS servisleriyle güvenli bir şekilde iletişim kurabilmesi için bir IAM Kullanıcısı (VideoAnalizBot) oluşturulmuştur.

- Uygulamanın videoları buluta yükleyebilmesi için **AmazonS3FullAccess**, videoları yapay zekaya izletebilmesi için ise **AmazonRekognitionFullAccess** politikaları (policies) bu kullanıcıya atanmıştır.

- Siber güvenlik prensipleri geređi, dıřarıdan erişimi sağlamak adına bir "Access Key" ve "Secret Access Key" çifti üretilmiş, bu anahtarlar kaynak kodlara (hardcoded) yazılmak yerine güvenli bir şekilde saklanmıştır.

3. Bulut Depolama Alanının (Amazon S3) Hazırlanması Yapay zeka servisi olan Rekognition'ın videoları analiz edebilmesi için öncelikle bu videoların bulutta depolanması gerekmektedir.

- AWS Console üzerinden us-east-1 (N. Virginia) bölgesinde, berkay-guvenlik-videolari-2026 adında benzersiz bir S3 Kovası (Bucket) oluşturulmuştur. Bölge seçiminde, yapay zeka servislerinin en stabil ve hızlı çalıştığı merkezlerden biri olması dikkate alınmıştır.

4. Bağımlılıkların ve Güvenli Kasa (.env) Altyapısının Kurulması Proje dizini oluşturulduktan sonra, Python'ın AWS ile haberleşmesini sağlayacak olan resmi kütüphane (boto3) ve şifreleri gizleyecek olan kütüphane (python-dotenv) terminal üzerinden projeye dahil edilmiştir:

- pip install boto3 python-dotenv
- Proje ana dizininde bir .env dosyası oluşturularak, AWS'den alınan Access Key ve Secret Key bilgileri bu dosyaya yerleştirilmiştir. Böylece kimlik bilgilerinin GitHub gibi açık kaynak platformlara sızması engellenmiştir.

BÖLÜM 2: CORE BACKEND VE AWS BULUT SERVİS ENTEGRASYONLARININ KODLANMASI

Projenin bu aşamasında, yerel ortamda çalışan Python kodunun AWS bulut ekosistemiyle entegrasyonu sağlanmış; asenkron veri işleme boru hattı (Data Pipeline) ve yapay zeka analiz motoru inşa edilmiştir.

1. AWS S3 Nesne Yükleme Mekanizması Yerel güvenlik kamerası kayıtlarının (test_videosu.mp4) analiz edilebilmesi için öncelikle bu dosyaların bulut nesne depolama alanına aktarılması gerekir.

- **Boto3 İstemcisi:** .env dosyasından okunan güvenli erişim anahtarları kullanılarak bir boto3.client('s3') nesnesi ayağa kaldırılmıştır.
- **Hata Yönetimi ve Validasyon:** İlk testlerde, AWS S3 adlandırma standartlarının (Regex kısıtlamaları) dışına çıkan ufak bir karakter boşluğu (whitespace) taranmış ve parametre doğrulama hatası (Parameter validation failed) başarıyla çözülerek kararlı bir yükleme mimarisi elde edilmiştir.
- **upload_file Operasyonu:** Kod bloğu içerisindeki yerel dosya yolu, hedef kova adı ve buluttaki kayıt ismi parametreleri s3_client.upload_file() metoduna asenkron olarak geçilerek videonun AWS altyapısına kayıpsız transferi doğrulanmıştır.

2. AWS Rekognition Asenkron Video Analiz Motoru Görüntü işlemede anlık fotoğrafların aksine, videolar zamana bağlı (Time-series) akışlar içerdiği için senkron bir istek-cevap mimarisi sunucuyu kilitleyecektir. Bu tıkanıklığı (bottleneck) önlemek adına **Asenkron Polling (Sorgulama) Mimarisi** kurgulanmıştır.

- **start_label_detection:** Analiz süreci rekognition_client.start_label_detection() metodu ile başlatılır. Bu metot, AWS tarafında bir analiz işi (Job) tanımlar ve geriye benzersiz bir JobId değeri döndürür. Yazılımımız bu sayede

analizin bitmesini bloklanarak beklemek yerine, arka planda işin takibini sürdürür.

- **MinConfidence Filtresi:** Sistem doğruluğunu artırmak ve sahte pozitif (False Positive) uyarılardan kaçınmak amacıyla yapay zekanın emin olma eşik değeri %75 (MinConfidence=75) olarak kısıtlanmıştır.

3. Durum Sorgulama (Polling Loop) ve Veri İşleme Mantığı Bulut sunucularında videonun kare hızına (FPS) göre parçalanıp derin öğrenme modellerinden geçirilmesi süreci, kod içerisindeki dinamik bir while True döngüsüyle takip edilmiştir.

- **get_label_detection:** Döngünün her adımında rekognition_client.get_label_detection(JobId=job_id) metodu çağrılarak AWS'deki işin güncel durumu (JobStatus) kontrol edilmiştir.
- **Sistem Kaynak Yönetimi:** AWS API limitlerini aşmamak ve işlemciyi (CPU) gereksiz yere yormamak adına her sorgu arasına time.sleep(3) komutu eklenerek kararlı bir sorgu periyodu (3 saniyelik gecikme) oluşturulmuştur.
- **Analiz Sonuçlarının Yakalanması:** İşlem durumu SUCCEEDED (Başarılı) sinyali verdiğinde döngü kırılır. Yapay zeka modeli videodaki tüm nesneleri, ortam koşullarını ve canlıları detaylı bir JSON nesnesi halinde backend katmanına teslim eder.

4. Veri Optimizasyonu ve Benzersiz Veri Seti (Set) Mantığı AWS

Rekognition, videonun farklı saniyelerinde yakaladığı aynı nesneyi (örneğin her saniyede tekrar eden "Person" etiketini) defalarca raporlar. Bu durum uç noktada veri kirliliğine yol açtığı için backend mimarisinde bir optimizasyon katmanı tasarlanmıştır:

- Gelen ham veriler içerisindeki tekrar eden etiketleri temizlemek amacıyla Python'ın yerleşik veri yapısı olan benzersiz ve sırasız **set()** (Küme) mimarisi entegre edilmiştir.
- Bu sayede her bir etiket (Person, Adult, Male vb.) ve o etikete ait en yüksek güven oranı (Confidence Score) cımbızla çekilerek tekil hale getirilmiş ve temiz bir analiz raporu elde edilmiştir.

•

BÖLÜM 3: FRONTEND (FLASK WEB PANELİ) GELİŞTİRME VE İHLAL ALARM MİMARİSİ

Projenin son katmanında, arka planda çalışan yapay zeka analiz sürecini son kullanıcıya (güvenlik görevlisi veya sistem yöneticisi) görsel ve interaktif bir şekilde sunabilmek amacıyla **Flask** mikro web iskeleti kullanılarak bir kontrol paneli (Dashboard) tasarlanmıştır.

1. Web Sunucusu ve Rota (Routing) Yönetimi Python kodlarının sadece terminalde çalışmasından öteye geçilerek, uygulamanın bir web sunucusu (localhost:5000) üzerinden yayın yapması sağlanmıştır.

- **Ana Sayfa (/):** @app.route('/') rotası ile kullanıcılar sisteme ilk girdiklerinde index.html şablonu temiz bir başlangıç durumuyla (durum=False) render edilir.
- **Analiz Tetikleyici (/analiz-et):** Kullanıcı arayüzündeki butona tıkladığında, sistem bu rotaya bir POST isteği atar. Bu istek, Bölüm 2'de tasarladığımız AWS Rekognition asenkron analiz döngüsünü başlatır.

2. Kullanıcı Arayüzü (UI) Tasarımı Güvenlik sistemlerinin endüstriyel standartlarına uygun olarak, göz yormayan Karanlık Tema (Dark Mode) tabanlı bir arayüz kodlanmıştır.

- HTML5 ve CSS3 teknolojileri kullanılarak merkezi bir kapsayıcı (Container) oluşturulmuş, butonlar ve veri listeleri modern web tipografisine uygun olarak şekillendirilmiştir.
- Tasarım, templates klasörü altında index.html dosyası olarak konumlandırılarak Flask'ın şablon motoruyla entegre çalışacak hale getirilmiştir.

3. Dinamik Şablonlama (Jinja2) ve Olay Tetikleme Backend katmanında işlenen yapay zeka verilerinin HTML arayüzüne dinamik olarak basılması için Jinja2 şablonlama motoru kullanılmıştır.

- **Şartlı Alarm (If Condition):** Backend tarafındaki döngüde, AWS'den dönen etiketler arasında isim == "Person" eşleşmesi yakalandığında is_person_detected bayrağı (flag) True değerine çekilir. HTML tarafındaki {% if durum %} bloğu bu değeri dinler ve eğer True ise ekranda kırmızı arka planlı, belirgin bir "**DİKKAT: İZİNSİZ GİRİŞ TESPİT EDİLDİ!**" uyarı kutusunu (Alert) görünür hale getirir.
- **Veri Döngüsü (For Loop):** Yapay zekanın tespit ettiği diğer tüm nesneler ve güven oranları (Confidence), {% for etiket in etiketler %} döngüsü ile sayfada maddeler (liste) halinde ekrana yazdırılır.

SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında; yerel ortamdaki video kayıtlarının Python (boto3) ile başarılı bir şekilde AWS S3 bulut depolama alanına aktarılması, AWS Rekognition API'si kullanılarak derin öğrenme tabanlı nesne tanıma ve etiketleme işlemlerinin gerçekleştirilmesi hedefleri başarıyla tamamlanmıştır. Bulut bilişim servislerinin birbirleriyle asenkron ve güvenli haberleşmesi sağlanarak, siber güvenlik prensiplerine (IAM, gizli anahtarlar) uygun, web tabanlı, uçtan uca bir "Akıllı İhlal Tespit Sistemi" ortaya konulmuştur.



Bulut Tabanlı İhlal Tespit Sistemi

AWS Rekognition ile güvenlik kamerası analizi

 Son Yüklenen Videoyu Analiz Et

 **DİKKAT: İZİNSİZ GİRİŞ TESPİT EDİLDİ!**
(Sistemde "İnsan" Algılandı)

 **AWS Yapay Zeka Raporu:**

- **Head** (Güven: %87.7)
- **Nature** (Güven: %100.0)
- **Outdoors** (Güven: %100.0)
- **Person** (Güven: %97.3)
- **Weather** (Güven: %100.0)
- **Face** (Güven: %79.2)
- **Adult** (Güven: %93.2)
- **Male** (Güven: %89.9)
- **Man** (Güven: %89.9)

```
1 AWS_ACCESS_KEY_ID=buraya_kendi_access_keyini_gir
2 AWS_SECRET_ACCESS_KEY=buraya_kendi_secret_keyini_gir
3 AWS_REGION=us-east-1
```

- Bu görselde, projenin siber güvenlik standartlarını sağlamak ve kimlik hırsızlığını önlemek amacıyla yapılandırılan çevresel değişken şablonu (.env.example) görülmektedir.
- Bulut bilişim mimarilerinde AWS erişim anahtarları (Access Key ve Secret Key) gibi hassas veriler, güvenlik zafiyeti yaratmamak adına kaynak kodların içerisine asla doğrudan (hardcoded) gömülmez.
- Projede yer alan gerçek .env dosyası, .gitignore mekanizması ile Git takibinden tamamen çıkarılarak açık kaynaklı platformlara (GitHub) sızması engellenmiş; bunun yerine sistemi kuracak diğer geliştiricilere rehberlik etmesi amacıyla bu güvenli şablon dosyası sisteme entegre edilmiştir.

```
load_dotenv()

app = Flask(__name__)

# AWS İstemcileri
s3_client = boto3.client('s3', region_name=os.getenv('AWS_REGION'))
rekognition_client = boto3.client('rekognition', region_name=os.getenv('AWS_REGION'))

BUCKET_ADIM = "berkay-guvenlik-videolari-2026" # BURAYI KENDİ BUCKET ADINLA DEĞİŞTİR!
S3_KAYIT_ADI = "kamera1_kayit.mp4"

@app.route('/')
def index():
```

Bu kod bloğu, Python arka plan (Backend) uygulamasının AWS bulut servisleriyle entegrasyonunu sağlayan ana yapılandırma alanıdır.

load_dotenv() fonksiyonu çağrılarak, bir önceki adımda oluşturulan .env dosyasındaki gizli AWS erişim anahtarları sistem belleğine güvenli bir şekilde yüklenmiştir.

AWS'nin resmi Python SDK'sı olan **Boto3** kütüphanesi kullanılarak iki farklı istemci (Client) ayağa kaldırılmıştır:

1. **s3_client**: Güvenlik kamerası kayıtlarını bulut depolama alanına yüklemekle görevlidir.
2. **rekognition_client**: S3'e yüklenen bu videoları asenkron olarak analiz edecek olan yapay zeka motoruna bağlanır.

Her iki istemci de yapılandırılırken region_name parametresi ile verilerin ABD Doğu (N. Virginia) veri merkezinde (us-east-1) işleneceği belirtilmiş ve BUCKET_ADIM değişkeniyle projenin hedef S3 kovalasına (berkay-guvenlik-videolari-2026) yönlendirme yapılmıştır.

Ayrıca app = Flask(__name__) satırı ile sistemin sadece bir terminal betiği (script) olmaktan çıkarılıp, tarayıcı üzerinden erişilebilecek bir web sunucusuna dönüştürülme temelleri atılmıştır.

```

<body>
  <div class="container">
    <h1>🔍 Bulut Tabanlı İhlal Tespit Sistemi</h1>
    <p>AWS Rekognition ile güvenlik kamerası analizi</p>

    <form action="/analiz-et" method="POST">
      <button type="submit" id="analizBtn">🚨 Son Yüklenen Videoyu Analiz Et</button>
    </form>

    {% if durum %}
      <div class="alert" style="display: block;">
        ⚠️ DİKKAT: İZİNSİZ GİRİŞ TESPİT EDİLDİ! <br> (Sistemde "İnsan" Algılandı)
      </div>

      <div id="sonuclar">
        <h3>🔍 AWS Yapay Zeka Raporu:</h3>
        <ul>
          {% for etiket in etiketler %}
            <li><b>{{ etiket.isim }}</b> (Güven: {{ etiket.guven }})</li>
          {% endfor %}
        </ul>
      </div>
    {% endif %}
  </div>
</body>

```

Bu görselde, güvenlik sisteminin ön yüzünü (Frontend) oluşturan HTML mimarisi ve **Jinja2** dinamik şablonlama motorunun (Templating Engine) kod yapısı görülmektedir.

Python backend katmanından (Flask üzerinden) gönderilen analiz sonuçları, statik HTML içerisine gömülmek yerine akıllı bloklarla işlenmektedir. `{% if durum %}` şart bloğu, yapay zekanın görüntüde "İnsan" (Person) tespit edip etmediğini kontrol eden ana tetikleyici mekanizmadır.

Backend'den gelen durum bayrağı (flag) True değerine ulaştığında, sistem anında reaksiyon göstererek gizlenmiş olan kırmızı renkli "İZİNSİZ GİRİŞ TESPİT EDİLDİ!" alarmını (display: block stiliyle) görünür hale getirir.

`{% for etiket in etiketler %}` iterasyonu (döngüsü) ise, AWS Rekognition API'sinden dönen çoklu tespit verilerini ve bu tespitlerin doğruluk oranlarını (Güven: %...) liste elemanları () olarak dinamik bir şekilde ekrana basar. Bu yapı, veri seti ne kadar büyük olursa olsun arayüzün esnek bir şekilde uyum sağlamasını garanti eder.